
1. What is Asynchronous Programming?

Asynchronous Programming is a **non-blocking execution model** that allows your application to perform multiple operations at the same time **without waiting** for long-running tasks (I/O operations) to complete.

In synchronous execution:

- One operation runs at a time
- The thread is blocked while waiting

In asynchronous execution:

- Operations start and **give back the thread**
- Once the operation completes, execution continues

2. Why Asynchronous Programming?

Long-running operations:

- Database Calls
- API Calls
- File I/O
- Network Operations
- Delays / Timers

If such operations run synchronously:

- The thread is blocked
- Application becomes unresponsive (UI freezes)
- Web server cannot serve multiple requests efficiently

Async execution:

- Frees the thread meanwhile
 - Improves throughput and scalability
-

4. Core Components: async & await

async keyword

- Marks a method as asynchronous
- Enables the use of await

await keyword

- Tells the compiler:
 - Pause execution of this method
 - Release the current thread
 - Resume when the awaited task completes

Example:

```
static async Task FetchDataAsync()
{
    Console.WriteLine("Fetching started...");
    await Task.Delay(3000); // Simulates a long operation
    Console.WriteLine("Fetching completed...");
}
```

Execution flow:

1. "Fetching started" prints immediately
2. await Task.Delay(3000) releases the thread
3. After 3 seconds, program resumes and prints "Fetching completed"

7. When to Use Asynchronous Programming?

Use async for:

- Web APIs
- GUI applications
- Networking

- DB queries
- File uploads/downloads

Do not use async for:

- Tight CPU loops
- Heavy computation

There, use:

```
Task.Run(() => HeavyCalculation());
```

9. Advantage Summary

Benefit	Description
Scalability	Server handles more requests
Performance	Less thread blocking
Responsiveness	UI remains interactive

Asynchronous programming in C#:

- Uses Task + async + await
- Frees threads during waiting
- Enables high-performance, scalable, responsive applications
- Especially beneficial for I/O bound operations

Important Rules of async/await

Rule	Explanation
Only async methods can use await	Compiler enforceable rule
async does not run parallel by default	You must start multiple tasks manually
async Task returns void-like asynchronous execution	No return value

Rule	Explanation
async Task<T> returns a result asynchronously	Wraps returned data
Avoid async void except for events	Hard to debug errors